

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PASHCHIMANCHAL CAMPUS

Object Oriented Analyis and Design
On
EventX - One Place For All Events

Submitted By:
Aashish Adhikari (PAS079BCT003)
Ashim Shrestha (PAS079BCT007)
Sagun Paudel (PAS079BCT032)
Saugat Adhikari (PAS079BCT036)

Submitted To:
Department of Electronics and Computer Engineering
Pashchimanchal Campus
Pokhara, Nepal

February 27, 2026

Candidate Objects Definition:

EventX is a calendar based college event management system where users can interact through well-defined functionalities.
Students first login to the system, then view the event calendar. They can also search and filter class events or club events, view event details, receive notifications about upcoming events and view class timetables.
Faculty admins authenticate themselves and create, update and delete class events, upload or update timetables.
Club Admins use the system to create, manage, and update club events, and publish event information to the calendar. Admins oversee the overall operation of the system by managing users and permissions, controlling access.
Super Admin oversees the overall system, manages user and permissions and controls all the club and class events.
The system allows to filter and search events, timetable management, and notification delivery, enabling smooth coordination of the academic and extracurricular activities the user interacts with.

Candidate Objects are listed as:

- EventX
- Calendar
- System
- Student
- Login
- Event calendar
- Class event
- Club event
- Notification
- Timetable
- Faculty admin
- Authentication
- Club admin
- Permission
- Access
- Super admin
- Filter
- Search

Rejected Candidate Objects and Rejection Criteria

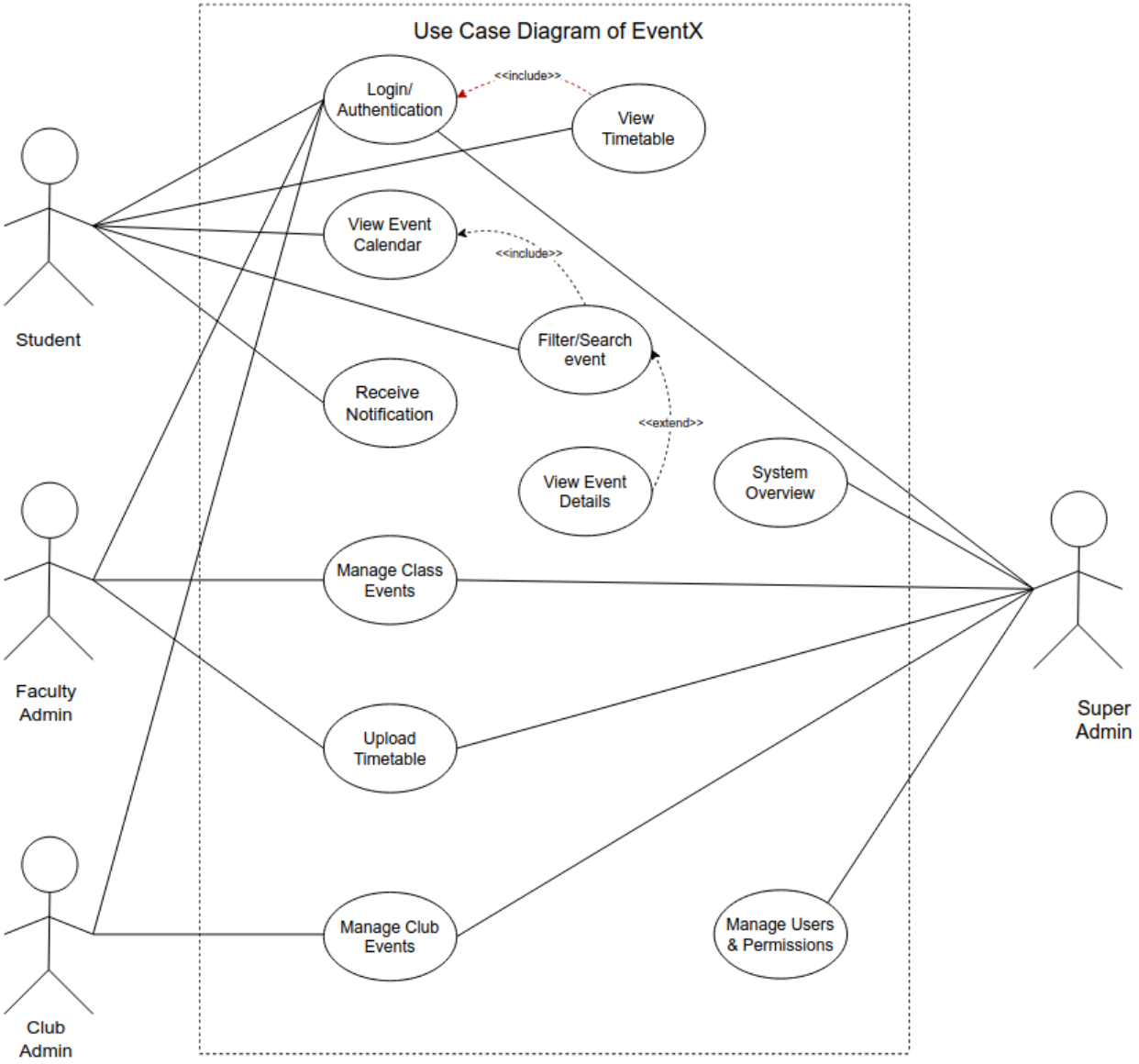
Candidate Objects	Rejection Criteria
EventX	too general (Represents the whole system, not a single object)
System	too general (Encompasses all components of EventX)
Login	Association (Represents an action performed by users)
Authentication	Association (Process of verifying users)
Filter, Search	Association (Functional operation on events)
Permission	Attribute (Describes access rights of users)
Access	Association (Represents a relationship between user and system)
Event calendar, calendar	Duplicate (Same meaning as calendar)
Club event, class event	Duplicate (Specialized form of event)

Final Objects (after rejection):

After applying rejection rules, the **final objects** in the EventX system are:

- Calendar
- Event
- Notification
- Timetable
- Student
- Faculty admin
- Club admin
- Super admin

Use Case Diagram:



Use Case UC1: Login / Authentication

Use Case ID	UC1
Use Case Name	Login / Authentication
Primary Actor	Student, Faculty Admin, Club Admin, Super Admin
Stakeholders & Interests	**Student:** Wants secure, quick access to events and timetables. **Faculty Admin:** Wants authenticated access to manage class events. **Club Admin:** Wants authenticated access to manage club events. **Super Admin:** Wants only authorised users to access the system.
Preconditions	User has a registered account in EventX.
Success Guarantee (Postconditions)	User is authenticated, a session is created, and the user is directed to their role-based dashboard.
Main Success Scenario	1. User opens the EventX login page. 2. User enters username/email and password. 3. System validates the credentials. 4. System identifies the user's role. 5. System creates a session and redirects the user to their dashboard.
Extensions (Alternative Flows)	**3a. Invalid credentials:** System shows an error; user may retry. **3b. Account locked:** Locked after 5 failed attempts; user must reset password. **2a. Forgot password:** System emails a reset link; user resets and logs in.
Special Requirements	- Passwords stored as salted hashes. - Session expires after 30 minutes of inactivity. - Login page must work on mobile devices.
Technology & Data Variations	- Credentials entered via web form or mobile app. - Future: institutional SSO/OAuth support.
Frequency of Occurrence	Multiple times daily per user.
Open Issues	- Should MFA be required for admin roles? - What is the minimum password complexity policy?

Use Case UC2: View Event Calendar

Use Case ID	UC2
Use Case Name	View Event Calendar
Primary Actor	Student
Stakeholders & Interests	**Student:** Wants all upcoming class and club events in one view. **Faculty Admin:** Wants class events visible to students after publishing. **Club Admin:** Wants club events to appear on the calendar promptly. **Super Admin:** Wants the calendar to always show accurate information.
Preconditions	User is logged in (UC1). At least one published event exists.
Success Guarantee (Postconditions)	Event calendar is displayed with all published events for the selected period.
Main Success Scenario	1. Student navigates to the Event Calendar. 2. System retrieves all published events for the current month. 3. System displays events in a monthly calendar view. 4. Student can navigate between months. 5. Student may search or filter events. 6. Student clicks an event to view its details.
Extensions (Alternative Flows)	**2a. No events for the period:** System displays 'No events found.' **4a. Student switches view:** System re-renders in weekly or daily mode. ***a. Data fails to load:** System shows an error with a retry option.
Special Requirements	- Calendar must be responsive on mobile screens. - Class and club events must be visually distinguished.
Technology & Data Variations	- Calendar rendered with a frontend library (e.g., FullCalendar.js). - Event data fetched via REST API.
Frequency of Occurrence	Multiple times daily by students.
Open Issues	- Should export to Google Calendar / iCal be supported? - How far ahead should events be visible?

Use Case UC3: Manage Class Events

Use Case ID	UC3
Use Case Name	Manage Class Events
Primary Actor	Faculty Admin
Stakeholders & Interests	**Faculty Admin:** Wants to create, edit, and delete class events easily. **Student:** Wants accurate class events on the calendar in real time. **Super Admin:** Wants class events properly categorised and auditable. **Institution:** Wants academic events coordinated to avoid conflicts.
Preconditions	Faculty Admin is logged in (UC1) and has class event management permissions.
Success Guarantee (Postconditions)	Class event is saved. Calendar is updated immediately. Students are notified.
Main Success Scenario	1. Faculty Admin opens 'Manage Class Events'. 2. System shows the list of existing class events. 3. Faculty Admin selects 'Create New Event'. 4. Faculty Admin fills in: title, date, time, venue, description, target audience. 5. System validates the input. 6. System publishes the event to the calendar. 7. System notifies relevant students.
Extensions (Alternative Flows)	**3a. Edit event:** Faculty Admin selects an event, edits fields, saves; students re-notified. **3b. Delete event:** Faculty Admin confirms deletion; event removed and students notified. **5a. Validation fails:** System highlights errors; Faculty Admin corrects and resubmits.
Special Requirements	- Description field must support rich text. - System must warn of venue/time conflicts with existing events.
Technology & Data Variations	- Form submission with client-side and server-side validation. - File attachments (slides, agendas) can be added to an event.
Frequency of Occurrence	Several times per week during the academic term.
Open Issues	- Can event visibility be restricted to specific student groups? - Is Super Admin approval required before publishing?

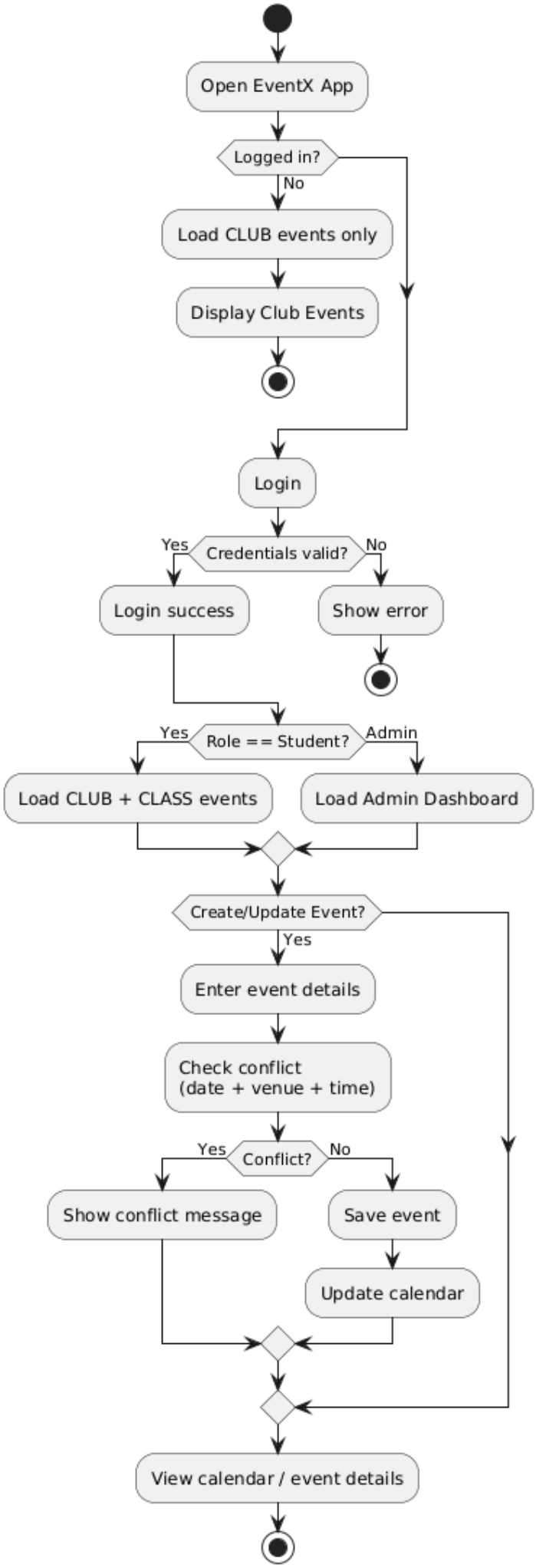
Use Case UC4: Manage Club Events

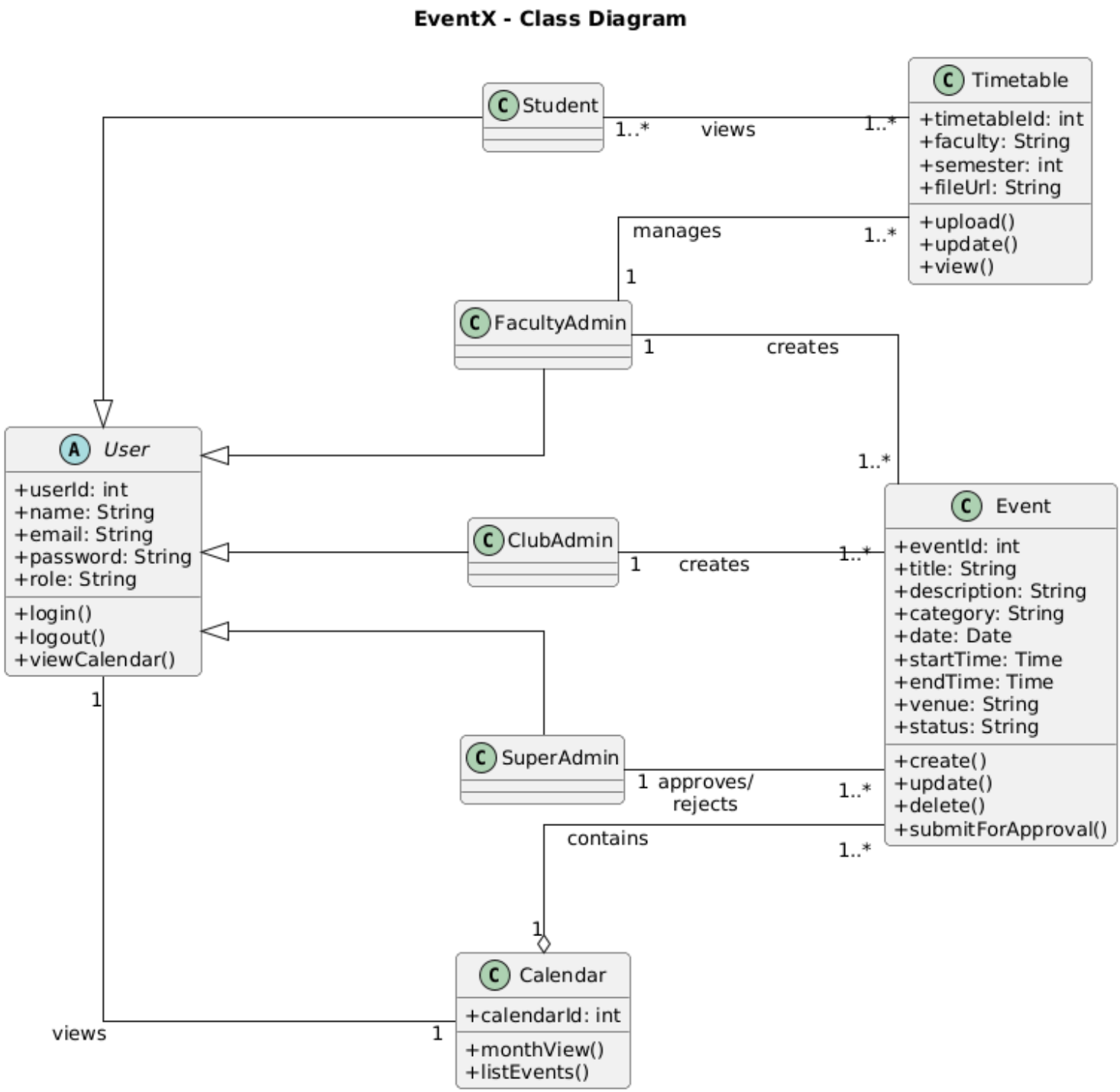
Use Case ID	UC4
Use Case Name	Manage Club Events
Primary Actor	Club Admin
Stakeholders & Interests	**Club Admin:** Wants to create and promote club events easily. **Student:** Wants to discover and attend relevant club events. **Super Admin:** Wants club events to comply with institutional policy before going live. **Institution:** Wants extracurricular events coordinated with academic ones.
Preconditions	Club Admin is logged in (UC1) and has permissions to manage their club's events.
Success Guarantee (Postconditions)	Club event is saved and published to the calendar. Students are notified.
Main Success Scenario	1. Club Admin opens 'Manage Club Events'. 2. System shows existing events for the admin's club. 3. Club Admin selects 'Create New Club Event'. 4. Club Admin fills in: title, date, time, venue, description, registration info. 5. System validates the input. 6. System publishes the event to the calendar. 7. System notifies students.
Extensions (Alternative Flows)	**3a. Edit event:** Club Admin modifies fields and saves; students re-notified. **3b. Delete event:** Club Admin confirms; event removed and participants notified. **6a. Approval required:** Event set to 'Pending'; Super Admin approves/rejects; Club Admin notified. **5a. Validation fails:** System highlights errors; Club Admin corrects and resubmits.
Special Requirements	- Club Admin can only manage events for their own club. - A venue is required before an event can be published.
Technology & Data Variations	- Rich text editor for descriptions. - Promotional images or documents can be attached.
Frequency of Occurrence	Several times per month during the academic year.
Open Issues	- Should Super Admin approval be mandatory or optional? - Can one Club Admin manage multiple clubs?

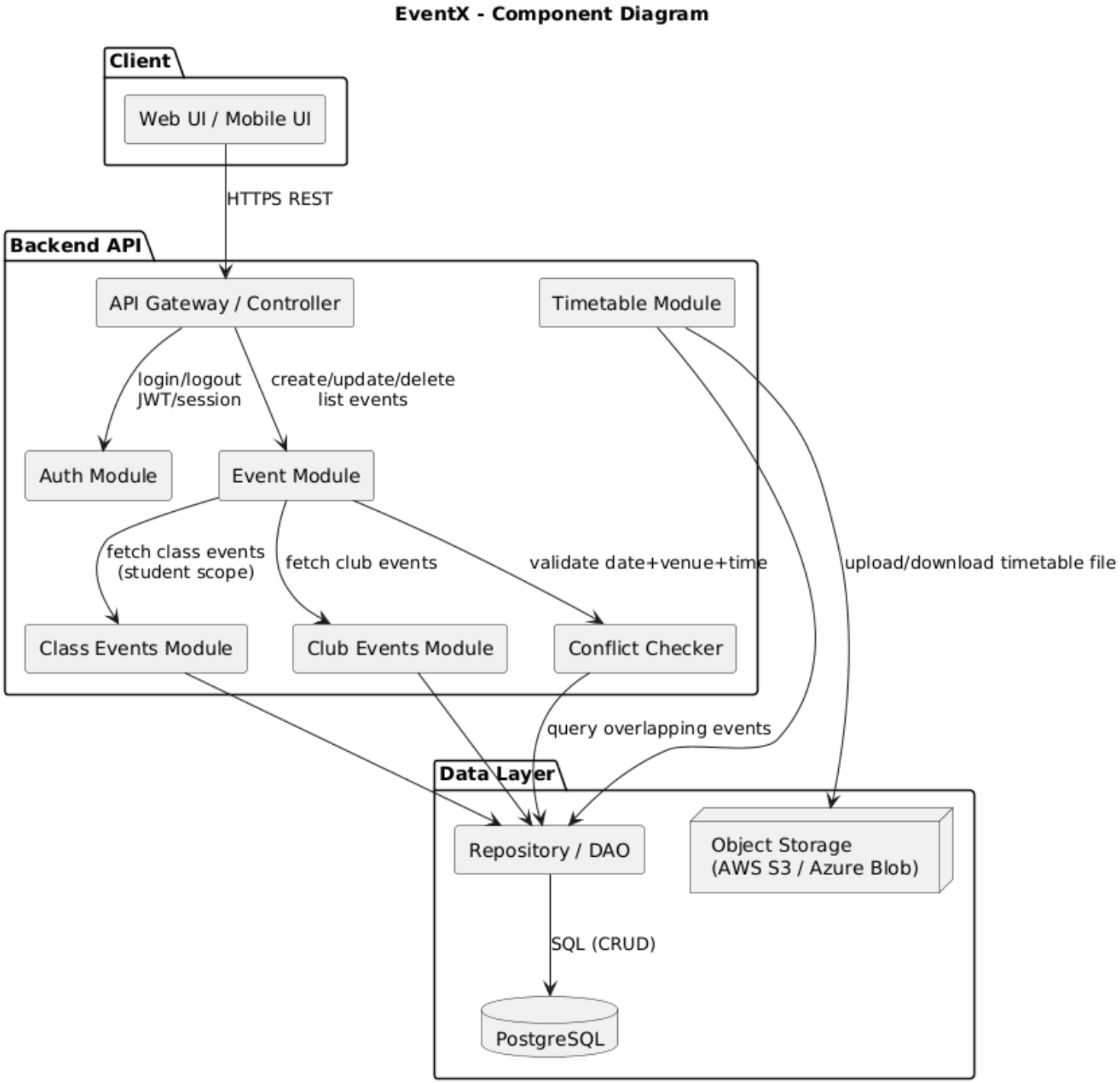
Use Case UC5: Manage Users & Permissions

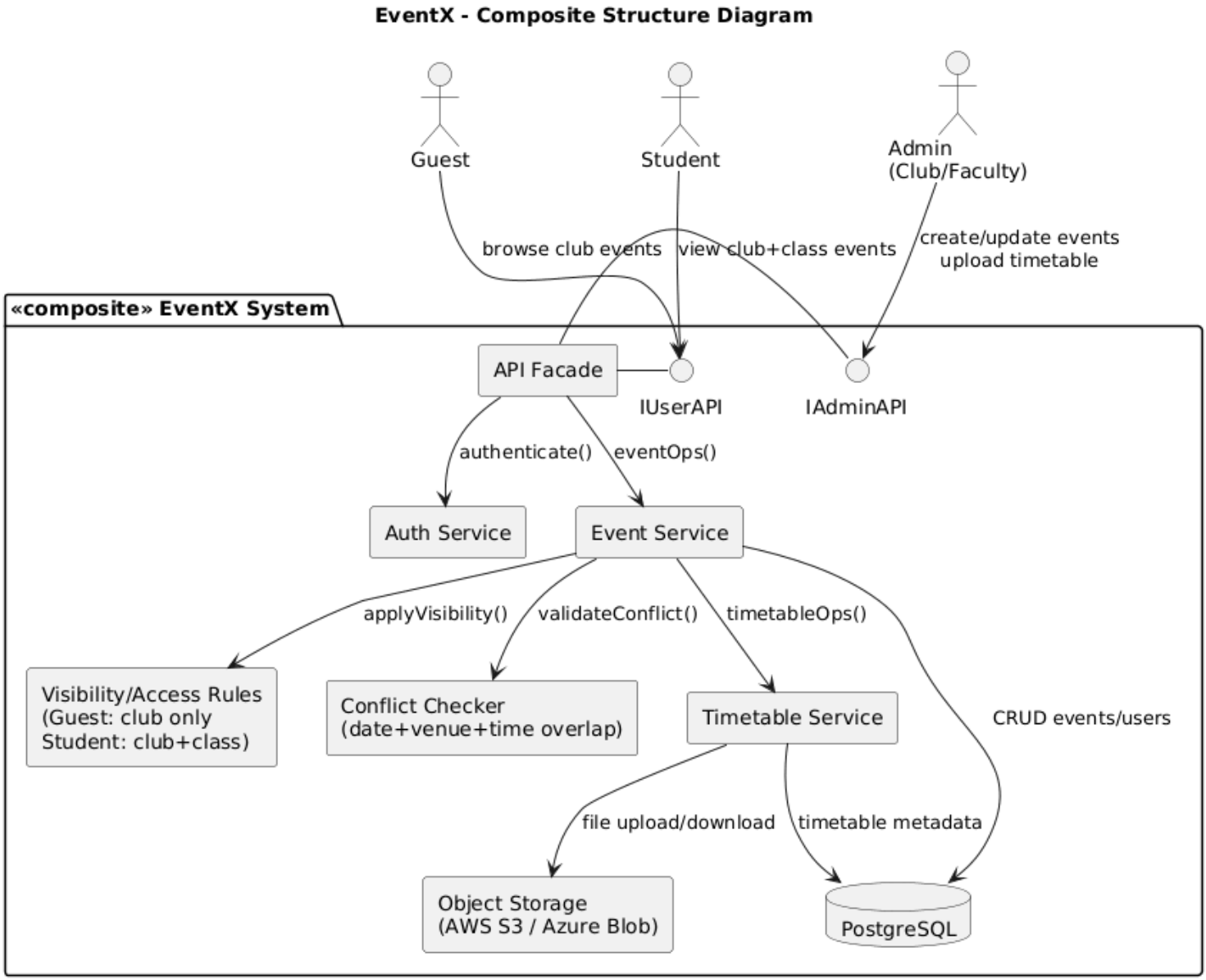
Use Case ID	UC5
Use Case Name	Manage Users & Permissions
Primary Actor	Super Admin
Stakeholders & Interests	**Super Admin:** Wants full control over user accounts and access rights. **Faculty Admin / Club Admin:** Want roles configured correctly to perform their tasks. **Student:** Wants their account active and correctly set up. **Institution:** Wants only authorised personnel in administrative roles.
Preconditions	Super Admin is logged in (UC1).
Success Guarantee (Postconditions)	User accounts are created, updated, or deactivated. Role changes take effect immediately.
Main Success Scenario	1. Super Admin opens 'User Management'. 2. System displays all users with their roles and status. 3. Super Admin selects a user or creates a new account. 4. Super Admin assigns a role (Student, Faculty Admin, Club Admin, Super Admin). 5. Super Admin sets or revokes specific permissions. 6. System saves changes; access rights update immediately. 7. System emails a welcome message if a new account was created.
Extensions (Alternative Flows)	**3a. Deactivate account:** System disables login and logs the action with a timestamp. **3b. Reactivate account:** System restores access for the selected user. **5a. Permission conflict:** System warns Super Admin; conflict must be resolved before saving. **3c. Bulk import:** Super Admin uploads a CSV; system creates accounts in batch and reports errors.
Special Requirements	- All permission changes must be logged for audit purposes. - Bulk import of student accounts via CSV must be supported. - Role changes must take effect without the user needing to re-login.
Technology & Data Variations	- Role-based access control (RBAC) enforced at the application layer. - Bulk import via CSV with server-side row validation.
Frequency of Occurrence	At the start of each semester and on-demand.
Open Issues	- Should multiple Super Admins be allowed, and how are their actions audited? - What is the process when a student is promoted to Club Admin?

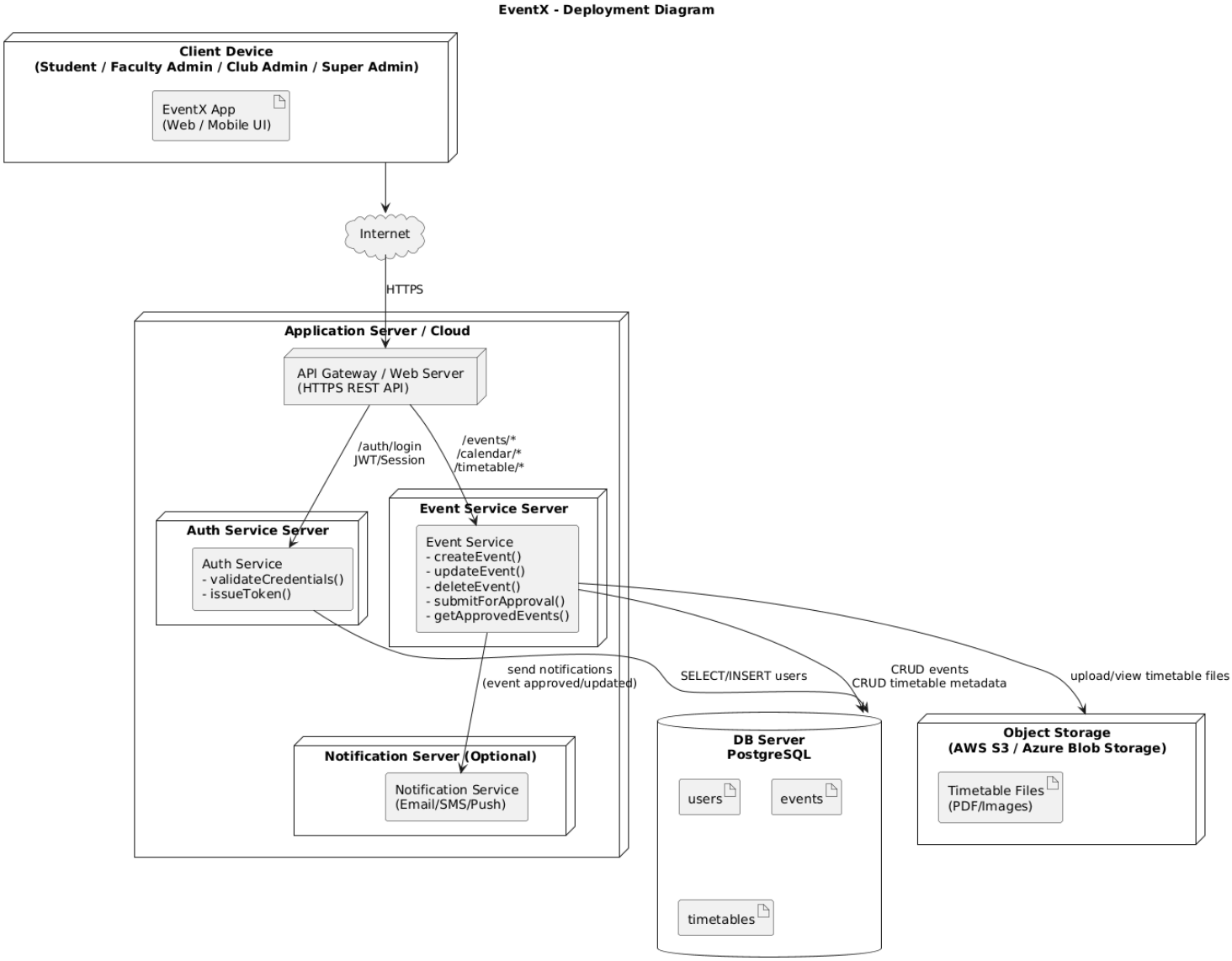
EventX - Activity Diagram

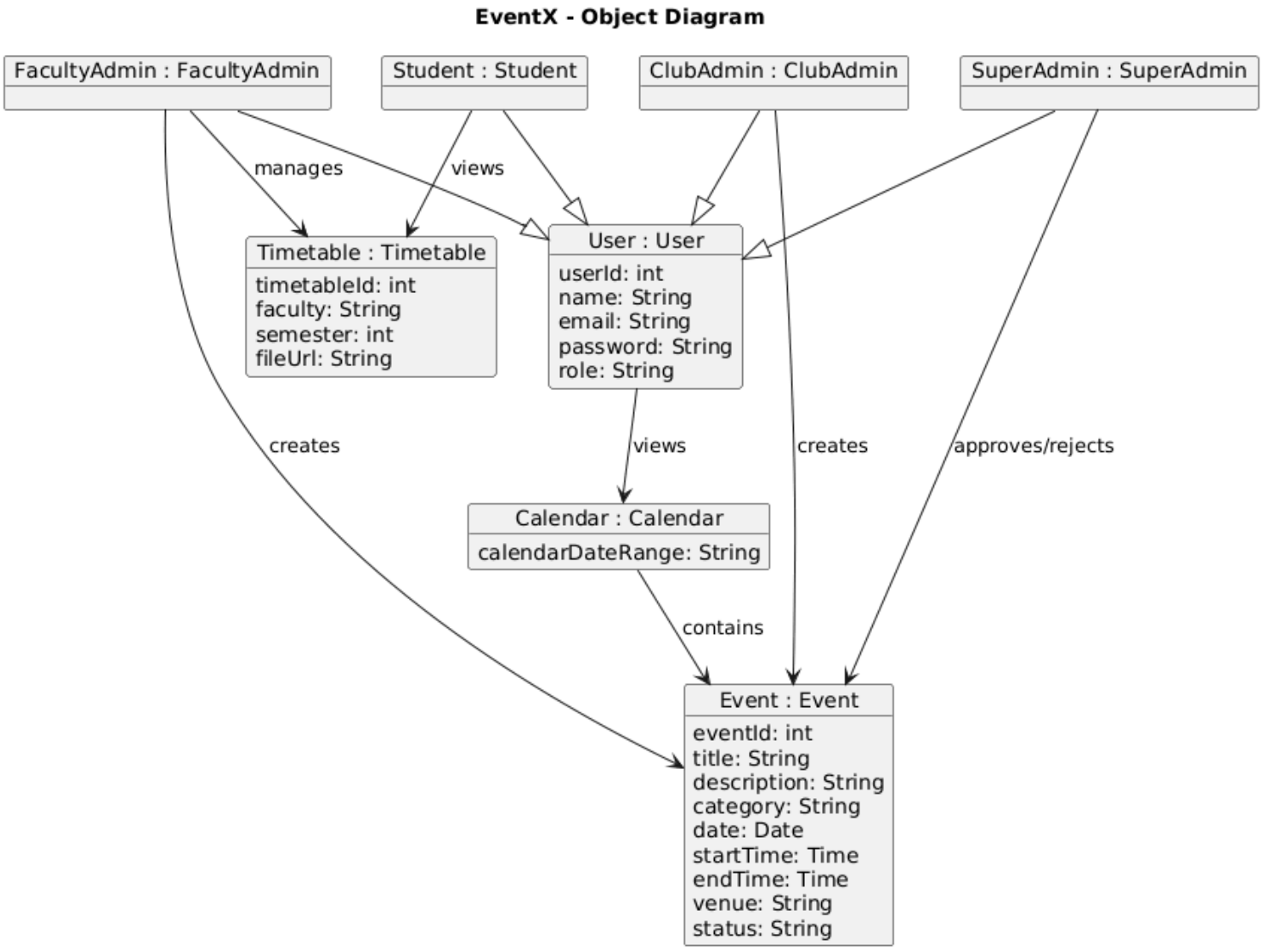


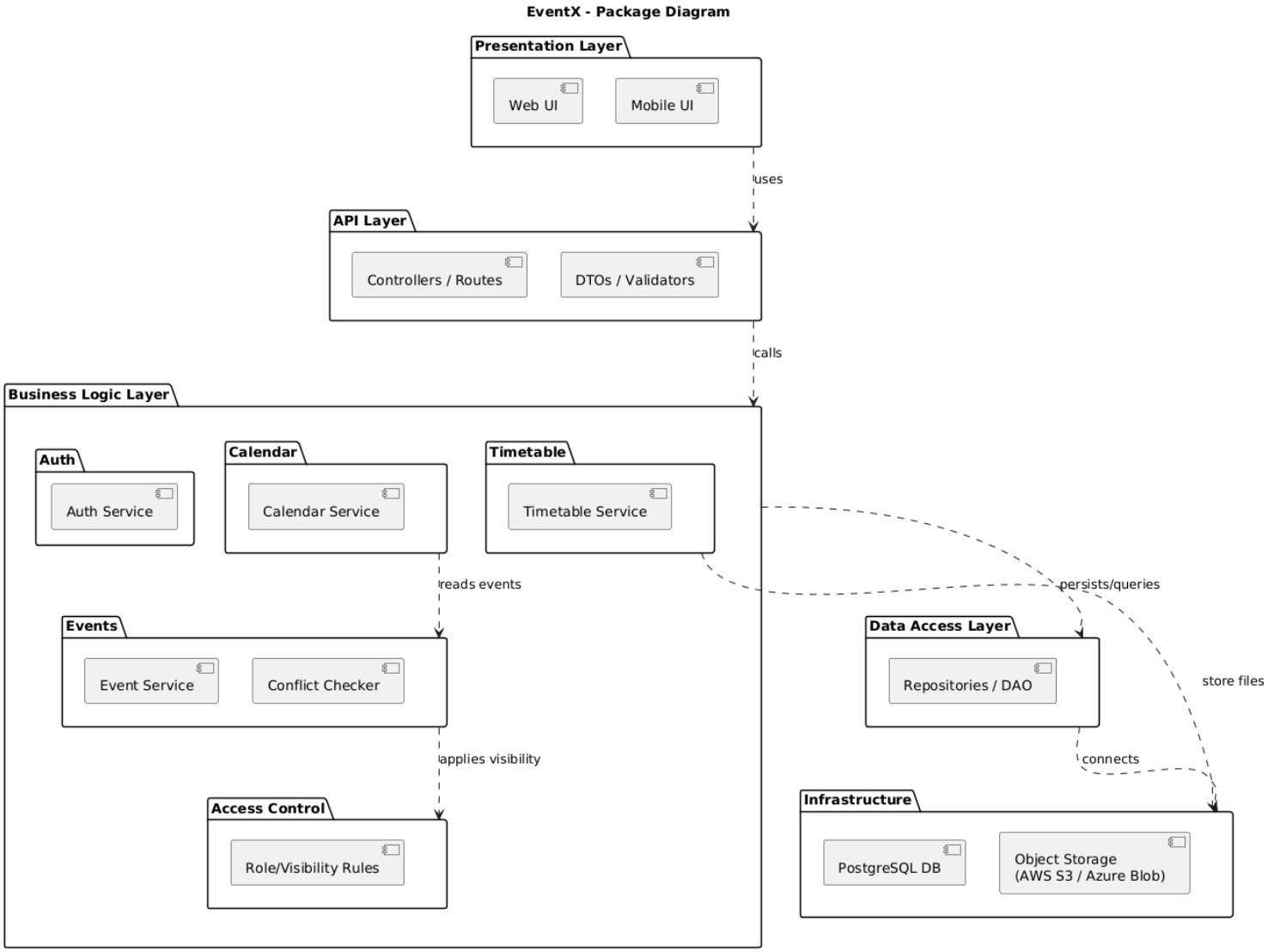

EventX - Class Diagram
Student
Timetable
+timetableId: int
+faculty: String
+semester: int
+fileUrl: String
+upload()
+update()
+view()
views
1..*
1..*
manages
1..*
1
FacultyAdmin
1
creates
1..*
User
+userId: int
+name: String
+email: String
+password: String
+role: String
+login()
+logout()
+viewCalendar()
ClubAdmin
1
creates
1..*
Event
+eventId: int
+title: String
+description: String
+category: String
+date: Date
+startTime: Time
+endTime: Time
+venue: String
+status: String
+create()
+update()
+delete()
+submitForApproval()
1
SuperAdmin
1 approves/
rejects
1..*
contains
1..*
1
Calendar
+calendarId: int
+monthView()
+listEvents()
views
1

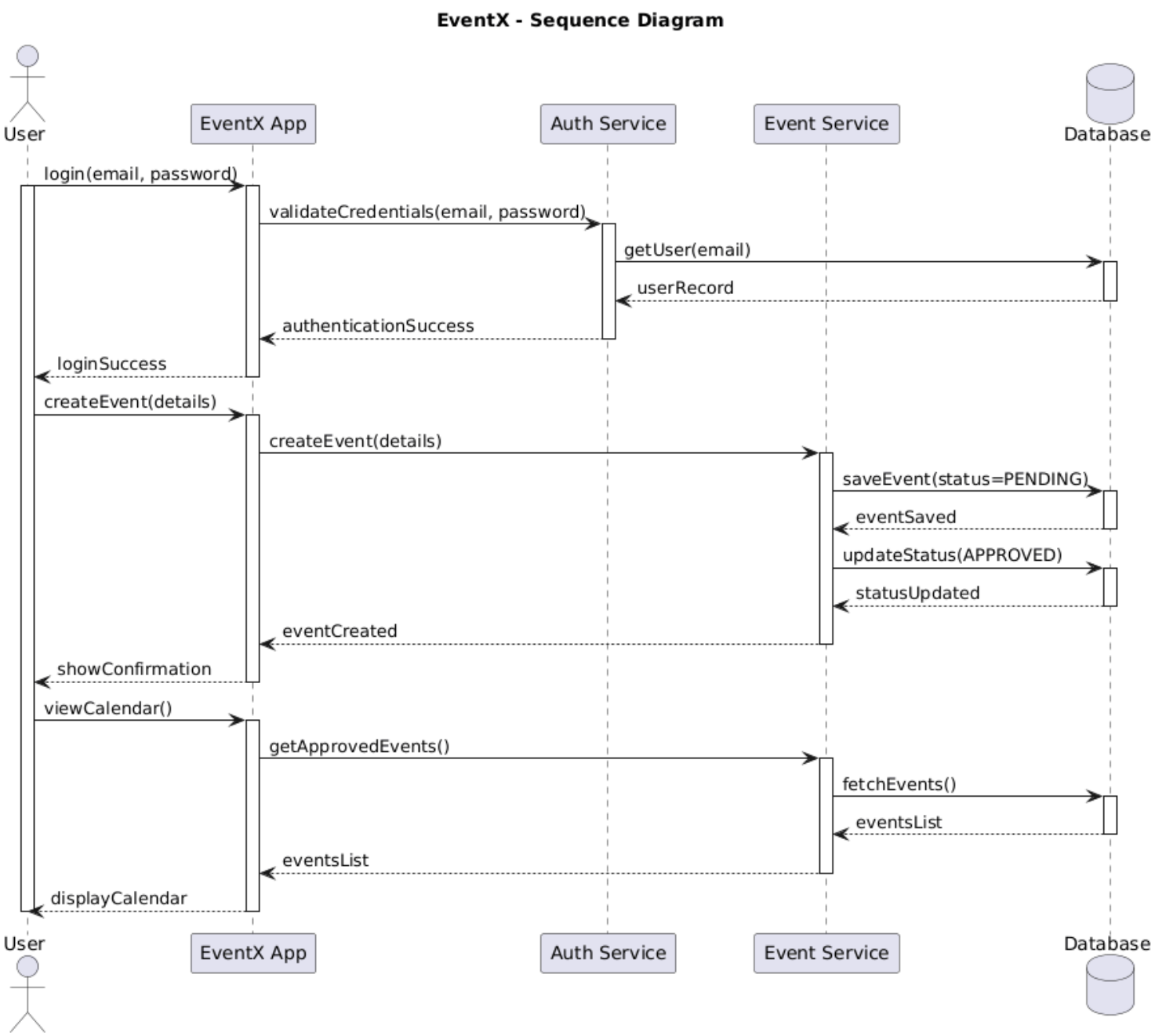

EventX - Component Diagram
Client
Web UI / Mobile UI
HTTPS REST
Backend API
API Gateway / Controller
Timetable Module
login/logout
JWT/session
create/update/delete
list events
Auth Module
Event Module
fetch class events
(student scope)
fetch club events
validate date+venue+time
upload/download timetable file
Class Events Module
Club Events Module
Conflict Checker
query overlapping events
Data Layer
Repository / DAO
Object Storage
(AWS S3 / Azure Blob)
SQL (CRUD)
PostgreSQL


EventX - Composite Structure Diagram
Guest
Student
Admin
(Club/Faculty)
browse club events
view club+class events
create/update events
upload timetable
«composite» EventX System
API Facade
IUserAPI
IAdminAPI
authenticate()
eventOps()
Auth Service
Event Service
applyVisibility()
validateConflict()
timetableOps()
Visibility/Access Rules
(Guest: club only
Student: club+class)
Conflict Checker
(date+venue+time overlap)
Timetable Service
CRUD events/users
file upload/download
timetable metadata
Object Storage
(AWS S3 / Azure Blob)
PostgreSQL


EventX - Deployment Diagram
Client Device
(Student / Faculty Admin / Club Admin / Super Admin)
EventX App
(Web / Mobile UI)
Internet
HTTPS
Application Server / Cloud
API Gateway / Web Server
(HTTPS REST API)
/auth/login
JWT/Session
/events/*
/calendar/*
/timetable/*
Auth Service Server
Auth Service
- validateCredentials()
- issueToken()
Event Service Server
Event Service
- createEvent()
- updateEvent()
- deleteEvent()
- submitForApproval()
- getApprovedEvents()
send notifications
(event approved/updated)
SELECT/INSERT users
CRUD events
CRUD timetable metadata
upload/view timetable files
Notification Server (Optional)
Notification Service
(Email/SMS/Push)
DB Server
PostgreSQL
users
events
timetables
Object Storage
(AWS S3 / Azure Blob Storage)
Timetable Files
(PDF/Images)


EventX - Object Diagram
FacultyAdmin : FacultyAdmin
Student : Student
ClubAdmin : ClubAdmin
SuperAdmin : SuperAdmin
manages
views
Timetable : Timetable
timetableId: int
faculty: String
semester: int
fileUrl: String
User : User
userId: int
name: String
email: String
password: String
role: String
creates
views
creates
approves/rejects
Calendar : Calendar
calendarDateRange: String
contains
Event : Event
eventId: int
title: String
description: String
category: String
date: Date
startTime: Time
endTime: Time
venue: String
status: String


EventX - Package Diagram
Presentation Layer
Web UI
Mobile UI
uses
API Layer
Controllers / Routes
DTOs / Validators
calls
Business Logic Layer
Auth
Auth Service
Calendar
Calendar Service
Timetable
Timetable Service
reads events
persists/queries
Events
Event Service
Conflict Checker
Data Access Layer
Repositories / DAO
store files
applies visibility
connects
Access Control
Role/Visibility Rules
Infrastructure
PostgreSQL DB
Object Storage
(AWS S3 / Azure Blob)


EventX - Sequence Diagram
User
EventX App
Auth Service
Event Service
Database
login(email, password)
validateCredentials(email, password)
getUser(email)
userRecord
authenticationSuccess
loginSuccess
createEvent(details)
createEvent(details)
saveEvent(status=PENDING)
eventSaved
updateStatus(APPROVED)
statusUpdated
eventCreated
showConfirmation
viewCalendar()
getApprovedEvents()
fetchEvents()
eventsList
eventsList
displayCalendar
User
EventX App
Auth Service
Event Service
Database

EventX - State Diagram

